USB chat coding bench

Model: `Deepseek Coder 6.7B (3.8 GB)` Started: 2026-10-06T10:20:01+00:00 Coding on, no GitHub. Vote opens only after host quality criteria pass (mean of trials). Loop tables update after every committee round.

Summary

- Tasks: 4. Finished: 4/4. Qualified: 0/4. Committee shipped: 0/4. Verifier passed: 0/4.

```
# | Task      | Loops | Qualified | Shipped | Verifier | Files
--+-----------+-------+-----------+---------+----------+------
1 | snake-dqn | 2     | no        | no      | fail     | 1
2 | maze-path | 2     | no        | no      | fail     | 1
3 | csv-sql   | 3     | no        | no      | fail     | 7
4 | expr-lang | 2     | no        | no      | fail     | 1
```

Loop progress (all tasks)

```
Task      | Loop | Qualified | Mean | Min | Scores | Files | Vote | Shipped
----------+------+-----------+------+-----+--------+-------+------+--------
snake-dqn | 1    | no        | -    | -   | -      | 1     | no   | no
snake-dqn | 2    | no        | -    | -   | -      | 1     | no   | no
maze-path | 1    | no        | -    | -   | -      | 1     | no   | no
maze-path | 2    | no        | -    | -   | -      | 1     | no   | no
csv-sql   | 1    | no        | -    | -   | -      | 7     | no   | no
csv-sql   | 2    | no        | -    | -   | -      | 7     | no   | no
csv-sql   | 3    | no        | -    | -   | -      | 7     | no   | no
expr-lang | 1    | no        | -    | -   | -      | 1     | no   | no
expr-lang | 2    | no        | -    | -   | -      | 1     | no   | no
```

1. Snake grid plus tiny DQN

- Id: `snake-dqn`
- Elapsed: 90.0 min
- Status: done
- Committee shipped: no
- Qualified: no
- Vote opened: no
- Vote: `(not opened)`
- Targets: `[{"id": "fruit_mean", "metric": "fruit_points", "n": 5, "aggregate": "mean", "op": ">=", "value": 16.0}, {"id": "fruit_min", "metric": "fruit_points", "n": 5, "aggregate": "min", "op": ">=", "value": 0.0}]`
- Scores: `{}`

- Qualify: fruit_mean: need 5 fruit_points values, got 0; fruit_min: need 5 fruit_points values, got 0
- Files: PROMPT.md
- Keep dir: `E:/outputs/2026-10-06_122024_snake-dqn`
- Coding session: `E:/coding-logs/2026-10-06_122024_snake-dqn`
- Coding log: `E:/coding-logs/2026-10-06_122024_snake-dqn/generic.jsonl`
- Verifier `test_snake.py`: fail (missing test_snake.py)

Loop table

```
Loop | Qualified | Mean | Min | Scores | Files | Vote | Shipped | Note
-----+-----------+------+-----+--------+-------+------+---------+-----------------------------
1    | no        | -    | -   | -      | 1     | no   | no      | fruit_mean: need 5 fruit_p..
2    | no        | -    | -   | -      | 1     | no   | no      | fruit_mean: need 5 fruit_p..
```

Prompt

Build a stdlib-only Snake grid and a tiny DQN. No pip, no numpy, no pygame.

Files (exactly these names):

- snake.py - 10x10 grid game. Four move directions (up, right, down, left). Reset starts a new game. A step that hits a wall or the snake body ends the game. Eating fruit scores +1 fruit point and the fruit respawns on a random empty cell.
- dqn.py - tiny neural net with nested Python lists, a replay buffer, epsilon-greedy action choice, and a train step. No third-party tensor libraries.
- train.py - trains the DQN for as many episodes as needed so eval games can score fruit.
- main.py - import the modules and call them with a short example (a few steps, not a full train) so `python main.py` finishes in a few seconds and exits 0. This is the usage spec the tester clones (different values).
- test_snake.py - the only test script.

Tester (test_snake.py):

- Import snake.py, dqn.py, and train.py. Clone the call pattern from main.py with different literals where that makes sense. Call the names those modules actually define. Do not invent other function or class names. Do not copy the game or the net into the test.
- In one process: train if the modules need it, then play 5 evaluation games with seeds 0, 1, 2, 3, 4. During eval, pick actions greedily (no exploration, no train step). Each game's score is fruit eaten that game.
- Also check that a reset yields a usable state and that walking into a wall ends the game.
- Never hardcode SCORES. Never assert or sys.exit before scores. Print one ASCII line:

SCORES fruit_points=a,b,c,d,e with the five eval fruit counts (not a total, no brackets).

- Execute timeout may be up to 600 seconds.

Quality (host qualify is the mean/min of those five values, not "at least one passed"):

- mean fruit_points >= 16
- min fruit_points >= 0

TARGETS: [{"id": "fruit_mean", "metric": "fruit_points", "n": 5, "aggregate": "mean", "op": ">=", "value": 16}, {"id": "fruit_min", "metric": "fruit_points", "n": 5, "aggregate": "min", "op": ">=", "value": 0}]

Committee reply (clipped)

```
Committee phase 1 round 2. Vote: (not opened)

Qualify: no (fruit_mean: need 5 fruit_points values, got 0; fruit_min: need 5 fruit_points values, got

Targets: [{"id": "fruit_mean", "metric": "fruit_points", "n": 5, "aggregate": "mean", "op": ">=", "val

Build a stdlib-only Snake grid and a tiny DQN. No pip, no numpy, no pygame.

Files (exactly these names):
- snake.py  -  10x10 grid game. Four move directions (up, right, down, left). Reset starts a new game.
- dqn.py  -  tiny neural net with nested Python lists, a replay buffer, epsilon-greedy action choice,
- train.py  -  trains the DQN for as many episodes as needed so eval games can score fruit.
- main.py  -  import the modules and call them with a short example (a few steps, not a full train) so

Files:
PROMPT.md

Seeded D:/Documents/Github Repositories/id10t/seeds/snake-dqn (1 files)
round 1 no vote (fruit_mean: need 5 fruit_points values, got 0; fruit_min: need 5 fruit_points values,

Coding session: E:/coding-logs/2026-10-06_122024_snake-dqn
```

Verifier stdout (clipped)

```

```

2. Maze shortest path

- Id: `maze-path`
- Elapsed: 90.0 min
- Status: done
- Committee shipped: no
- Qualified: no
- Vote opened: no

- Vote: `(not opened)`
- Targets: `[{"id": "maze_mean", "metric": "maze_ok", "n": 8, "aggregate": "mean", "op": ">=", "value": 1.0}]`
- Scores: `{}`
- Qualify: maze_mean: need 8 maze_ok values, got 0
- Files: PROMPT.md
- Keep dir: `E:/outputs/2026-10-06_135025_maze-path`
- Coding session: `E:/coding-logs/2026-10-06_135025_maze-path`
- Coding log: `E:/coding-logs/2026-10-06_135025_maze-path/generic.jsonl`
- Verifier `test_maze.py`: fail (missing test_maze.py)

Loop table

```
Loop | Qualified | Mean | Min | Scores | Files | Vote | Shipped | Note
-----+-----------+------+-----+--------+-------+------+---------+-----------------------------
1    | no        | -    | -   | -      | 1     | no   | no      | maze_mean: need 8 maze_ok ..
2    | no        | -    | -   | -      | 1     | no   | no      | maze_mean: need 8 maze_ok ..
```

Prompt

Build a stdlib-only maze generator and shortest-path solver. No pip.

Files (exactly these names):

- maze.py - make a grid with walls, a start cell, and a goal cell. Find a shortest path as a list of (row, column) cells from start to goal, or an empty list when there is no path. The test must be able to import this module and call whatever public names you define.
- main.py - import maze and call it with a small example so `python main.py` exits 0. This is the usage spec the tester clones (different values).
- test_maze.py - the only test script.

Tester (test_maze.py):

- Import maze.py. Clone the call pattern from main.py with different literals. Call the names that module actually defines. Do not invent other functions. Do not reimplement the generator or solver inside the test except for a small independent shortest-path check used only as an oracle.
- Run at least 8 trials in one loop:

1. Fixed open corridor: returned path length matches the oracle. 2. Sealed / unsolvable maze: return is empty. 3. Start cell is open. 4. Goal cell is open. 5-8. At least four seeded random mazes: a returned path stays in bounds, does not walk through walls, connects start to goal, and its length matches the oracle. An unsolvable fixture still returns empty.

- Each trial scores maze_ok 1 or 0 from the impl return value. Never hardcode SCORES. Never assert or sys.exit before scores. Print:

SCORES maze_ok=v1,v2,... with all trial values (no brackets, not a total).

- Finish in under 30 seconds.

Quality: mean maze_ok >= 1 (every trial must pass).

TARGETS: [{"id": "maze_mean", "metric": "maze_ok", "n": 8, "aggregate": "mean", "op": ">=", "value": 1}]

Committee reply (clipped)

```
Committee phase 1 round 2. Vote: (not opened)

Qualify: no (maze_mean: need 8 maze_ok values, got 0)

Targets: [{"id": "maze_mean", "metric": "maze_ok", "n": 8, "aggregate": "mean", "op": ">=", "value": 1

Build a stdlib-only maze generator and shortest-path solver. No pip.

Files (exactly these names):
- maze.py  -  make a grid with walls, a start cell, and a goal cell. Find a shortest path as a list of
- main.py  -  import maze and call it with a small example so ‘python main.py‘ exits 0. This is the us
- test_maze.py  -  the only test script.

Tester (test_maze.py):
- Import maze.py. Clone the call pattern from main.py with different literals. Call the names that mod

Files:
PROMPT.md

Seeded D:/Documents/Github Repositories/id10t/seeds/maze-path (1 files)
round 1 no vote (maze_mean: need 8 maze_ok values, got 0)

Coding session: E:/coding-logs/2026-10-06_135025_maze-path
```

Verifier stdout (clipped)

```

```

3. Tiny CSV SELECT

- Id: `csv-sql`
- Elapsed: 90.0 min
- Status: done
- Committee shipped: no
- Qualified: no
- Vote opened: no

- Vote: `(not opened)`
- Targets: `[{"id": "csv_mean", "metric": "query_ok", "n": 6, "aggregate": "mean", "op": ">=", "value": 1.0}]`
- Scores: `{}`
- Qualify: csv_mean: need 6 query_ok values, got 0
- Files: data/empty.csv

data/people.csv data/quoted.csv data/sample.csv data/ties.csv data/wide.csv PROMPT.md

- Keep dir: `E:/outputs/2026-10-06_152026_csv-sql`
- Coding session: `E:/coding-logs/2026-10-06_152026_csv-sql`
- Coding log: `E:/coding-logs/2026-10-06_152026_csv-sql/generic.jsonl`
- Verifier `test_csvsql.py`: fail (missing test_csvsql.py)

Loop table

```
Loop | Qualified | Mean | Min | Scores | Files | Vote | Shipped | Note
-----+-----------+------+-----+--------+-------+------+---------+----------------------------
1    | no        | -    | -   | -      | 7     | no   | no      | csv_mean: need 6 query_ok ..
2    | no        | -    | -   | -      | 7     | no   | no      | csv_mean: need 6 query_ok ..
3    | no        | -    | -   | -      | 7     | no   | no      | csv_mean: need 6 query_ok ..
```

Prompt

Build a stdlib-only CSV query helper (csv module is fine). No pip, no SQL engine.

Files (exactly these names):

- csvsql.py - read CSV files from data/. Support: pick a subset of columns; keep rows where one column equals a value; sort by a column ascending with a stable order on ties. If a named column is missing, either raise ValueError or return no rows - pick one rule and keep it. Quoted fields that contain a comma are one cell. An empty body yields no rows.
- main.py - import csvsql and call it on one existing data/*.csv file so `python main.py` exits 0. This is the usage spec the tester clones (different paths and column/filter/sort literals). Do not embed CSV text in Python strings.
- test_csvsql.py - the only test script.
- data/sample.csv - mixed depts; Ada and Bob share score 9.
- data/people.csv - extra ordinary rows.
- data/ties.csv - several rows share the same sort key.
- data/quoted.csv - a quoted field that contains a comma.
- data/empty.csv - empty body (no rows).
- data/wide.csv - extra columns for a subset trial.

Tester (test_csvsql.py):

- Import csvsql.py. Clone the call pattern from main.py with different literals. Call the names that module actually defines. Do not invent other functions. Do not reimplement filtering or sorting in the test.
- Use only the CSV files already in data/. Do not build CSV strings. Do not create new CSV files.
- Run at least 6 trials in one loop, each on an existing data/*.csv:

1. Subset of columns (wide.csv is for this). 2. Equality filter on a column. 3. Ascending sort. 4. Stable order when sort keys tie (ties.csv). 5. Missing column follows the chosen rule. 6. Quoted comma field parses as one cell (quoted.csv), and empty body yields no rows (empty.csv) - two checks in one trial or split, as long as n>=6.

- Each trial scores query_ok 1 or 0 from the impl return value. Never hardcode SCORES. Never assert or sys.exit before scores. Print:

SCORES query_ok=v1,v2,...

- Finish in under 30 seconds.

Quality: mean query_ok >= 1.

TARGETS: [{"id": "csv_mean", "metric": "query_ok", "n": 6, "aggregate": "mean", "op": ">=", "value": 1}]

Committee reply (clipped)

```
Committee phase 1 round 3. Vote: (not opened)

Qualify: no (csv_mean: need 6 query_ok values, got 0)

Targets: [{"id": "csv_mean", "metric": "query_ok", "n": 6, "aggregate": "mean", "op": ">=", "value": 1

Build a stdlib-only CSV query helper (csv module is fine). No pip, no SQL engine.

Files (exactly these names):
- csvsql.py  -  read CSV files from data/. Support: pick a subset of columns; keep rows where one colu
- main.py  -  import csvsql and call it on one existing data/*.csv file so ‘python main.py‘ exits 0. T
- test_csvsql.py  -  the only test script.
- data/sample.csv  -  mixed depts; Ada and Bob share score 9.

Files:
data/empty.csv
data/people.csv
data/quoted.csv
data/sample.csv
data/ties.csv
data/wide.csv
PROMPT.md

Seeded D:/Documents/Github Repositories/id10t/seeds/csv-sql (7 files)
round 1 no vote (csv_mean: need 6 query_ok values, got 0)
round 2 no vote (csv_mean: need 6 query_ok values, got 0)
```

```
Coding session: E:/coding-logs/2026-10-06_152026_csv-sql
```

Verifier stdout (clipped)

```
```

4. Tiny expression language

- Id: `expr-lang`
- Elapsed: 90.0 min
- Status: done
- Committee shipped: no
- Qualified: no
- Vote opened: no
- Vote: `(not opened)`
- Targets: `[{"id": "expr_mean", "metric": "expr_ok", "n": 8, "aggregate": "mean", "op": ">=", "value": 1.0}]`
- Scores: `{}`
- Qualify: expr_mean: need 8 expr_ok values, got 0
- Files: PROMPT.md
- Keep dir: `E:/outputs/2026-10-06_165027_expr-lang`
- Coding session: `E:/coding-logs/2026-10-06_165027_expr-lang`
- Coding log: `E:/coding-logs/2026-10-06_165027_expr-lang/generic.jsonl`
- Verifier `test_expr.py`: fail (missing test_expr.py)

Loop table

```
Loop | Qualified | Mean | Min | Scores | Files | Vote | Shipped | Note
-----+-----------+------+-----+--------+-------+------+---------+-----------------------------
1    | no        | -    | -   | -      | 1     | no   | no      | expr_mean: need 8 expr_ok ..
2    | no        | -    | -   | -      | 1     | no   | no      | expr_mean: need 8 expr_ok ..
```

Prompt

Build a stdlib-only arithmetic expression evaluator. No pip.

Files (exactly these names):

- expr.py - parse and evaluate + - * / and parentheses on floats. Multiplication and division bind tighter than addition and subtraction. Operators of the same precedence associate left to right (so 10-3-2 is 5). Invalid input raises ValueError. The test must import this module and call whatever public names you define.
- main.py - import expr and call it with a few example expressions so `python main.py` exits 0. This is the usage spec the tester clones (different values).

- test_expr.py - the only test script.

Tester (test_expr.py):

- Import expr.py. Clone the call pattern from main.py with different literals. Call the names that module actually defines. Do not invent other functions. Do not evaluate the expression with eval() or by copying the parser into the test except for one oracle that uses the ast module on well-formed extra cases.
- Run at least 8 trials in one loop. A trial scores expr_ok 1 only if the behavior matches:

1. 1+2*3 equals 7 2. (1+2)*3 equals 9 3. 10/4 equals 2.5 4. the text "nope" raises ValueError 5. 10-3-2 equals 5 6. (2+3)*(4-1) equals 15 7. whitespace around tokens is accepted for a well-formed expression 8. a random well-formed expression matches Python ast within 1e-9

- Each trial scores expr_ok 1 or 0 from the impl return value. Never hardcode SCORES. Never assert or sys.exit before scores. Print:

SCORES expr_ok=v1,v2,...

- Finish in under 30 seconds.

Quality: mean expr_ok >= 1.

TARGETS: [{"id": "expr_mean", "metric": "expr_ok", "n": 8, "aggregate": "mean", "op": ">=", "value": 1}]

Committee reply (clipped)

```
Committee phase 1 round 2. Vote: (not opened)

Qualify: no (expr_mean: need 8 expr_ok values, got 0)

Targets: [{"id": "expr_mean", "metric": "expr_ok", "n": 8, "aggregate": "mean", "op": ">=", "value": 1

Build a stdlib-only arithmetic expression evaluator. No pip.

Files (exactly these names):
- expr.py  -  parse and evaluate + - * / and parentheses on floats. Multiplication and division bind t
- main.py  -  import expr and call it with a few example expressions so ‘python main.py‘ exits 0. This
- test_expr.py  -  the only test script.

Tester (test_expr.py):
- Import expr.py. Clone the call pattern from main.py with different literals. Call the names that mod

Files:
PROMPT.md

Seeded D:/Documents/Github Repositories/id10t/seeds/expr-lang (1 files)
round 1 no vote (expr_mean: need 8 expr_ok values, got 0)

Coding session: E:/coding-logs/2026-10-06_165027_expr-lang
```

Verifier stdout (clipped)